FUNDOSCOPY STUDY · AI 학습자료

Mamba & Vision Mamba 스텝바이스텝 워크북

ViT만 알아도 따라오는 10단계 · 수식과 숫자로 이해 · 연습문제 + 해설

목표: "Attention은 O(L²), Mamba는 O(L)"를 수식·계산·코드로 스스로 설명할 수 있게 된다.

학습 로드맵

Step	주제	한 줄 목표
1	SSM이란	시퀀스를 "요약 메모리"로 처리하는 발상 이해
2	이산화와 Δ	$\bar{A} = e^{\Delta A}$, $\bar{B} \approx \Delta B$ 직접 계산
3	재귀 계산	상태 h_t 를 손으로 갱신
4	컨볼루션·커널	$K_k = C\bar{A}^k\bar{B}$ 유도하고 검산
5	선택성(S6)	입력이 Δ,B,C를 정한다는 것 이해
6	병렬 스캔	합성 규칙과 결합법칙으로 병렬화
7	Mamba 블록	conv·SSM·gate가 하는 일
8	Vision Mamba	패치화 + 양방향 스캔
9	복잡도	L vs L² 비용을 숫자로 비교
10	안저 적용	하이퍼파라미터로 CPU 모델 구성

사용법: 각 Step의 "확인 문제"를 먼저 풀고, 부록 A의 연습문제로 복습한 뒤 부록 B 해설로 채점하세요. 수식은 검은 글씨, 계산은 **직접 손으로** 하는 것을 권합니다.

Step 1. SSM이란 — 시퀀스를 요약 메모리로

> **핵심 발상**: 토큰을 전부 저장(attention)하는 대신, **하나의 상태 벡터(메모리)**를 순서대로 갱신한다.

연속시간 상태공간모델

$$h'(t) = A\,h(t) + B\,x(t), \qquad y(t) = C\,h(t) + D\,x(t)$$

기호	차원	의미
$x(t)$	D	입력(토큰의 특징)
$h(t)$	N	**은닉 상태 = 지금까지 정보의 압축**
A	N×N	상태 전이 (과거를 얼마나 넘길지)
B	N×D	입력 주입
C	D×N	출력 읽기
D	D	지름길(skip)

ViT 대비: attention은 "모든 토큰을 저장하고 매번 전부 조회"(O(L²)), SSM은 "상태를 순차 갱신"(O(L)). 정보를 **압축**한다는 점이 장점이자 한계(정보 병목).

연습 1 $h'(t) = Ah(t) + Bx(t)$ 에서 A, B, C 각각이 "기억·입력·출력" 중 무엇과 대응하는지 쓰고, A 가 영행렬이면 무슨 일이 생기는지 설명하시오. (부록 A 참고)

Step 2. 이산화와 Δ — 연속을 토큰 스텝으로

> **핵심**: 컴퓨터는 이산적이므로 스텝 크기 Δ 로 이산화한다.

$$\bar{A} = e^{\Delta A}, \qquad \bar{B} = \Big(\int_0^{\Delta} e^{\tau A}\, d\tau\Big) B = A^{-1}\big(e^{\Delta A} - I\big) B \;\approx\; \Delta B$$

$$h_t = \bar{A}\, h_{t-1} + \bar{B}\, x_t, \qquad y_t = C\, h_t + D\, x_t$$

Δ의 의미: 크면 **현재 입력을 강하게 반영**(과거는 잊음), 작으면 **상태를 오래 유지**.

> **안정성**: Mamba는 $A = -\exp(A_{\log})$ 로 두어 $A < 0 \to \bar{A} \in (0, 1)$ → 재귀가 **수축** → 기울기 안정.

예: $A = -1, B = C = 1$

Δ	$\bar{A} = e^{-\Delta}$	정확한 $\bar{B} = (1 - e^{-\Delta})$	근사 $\bar{B} = \Delta B$
0.5	0.6065	0.3935	0.5000
2.0	0.1353	0.8647	2.0000

> 연습 2 $A = -2,\ B = 0.5,\ \Delta = 1$ 일 때 $\bar{A}$, 정확한 $\bar{B}$, 근사 $\bar{B}$ 를 각각 소수 4자리로 계산하시오.

Step 3. 재귀 계산 — 상태를 손으로 갱신

> **핵심**: $h_t = \bar{A} h_{t-1} + \bar{B} x_t$ 를 토큰마다 반복.

$\bar{A} = 0.5,\ \bar{B} = 1,\ x = [2, 1, 0, 3]$, $h_0 = 0$ 으로 하면

t	x_t	계산	h_t
1	2	0.5·0 + 1·2	2.0
2	1	0.5·2 + 1·1	2.0
3	0	0.5·2 + 1·0	1.0
4	3	0.5·1 + 1·3	3.5

상태가 "과거를 압축"하지만 오래된 정보는 $\bar{A}^k$ 만큼 약해진다.

> **연습 3** $\bar{A} = 0.5,\ \bar{B} = 1,\ x = [1, 2, 3]$ 일 때 h_1, h_2, h_3 를 계산하시오.

Step 4. 컨볼루션과 커널 — 같은 식, 병렬 계산

> **핵심**: 재귀를 풀면 고정 커널과의 합성곱 $y = K * x$ 가 된다.

$$h_t = \sum_{s=1}^{t} \bar{A}^{\,t-s} \bar{B}\, x_s \;\Rightarrow\; y_t = \sum_{k=0}^{t-1} \underbrace{\left(C \bar{A}^{k} \bar{B}\right)}_{K_k} x_{t-k}$$

Step 3 예에서 $K_k = 0.5^k$: $K = [1,\ 0.5,\ 0.25,\ 0.125]$.

t	합성곱 계산	y_t
1	1·2	2.0
2	1·1 + 0.5·2	2.0
3	1·0 + 0.5·1 + 0.25·2	1.0
4	1·3 + 0.5·0 + 0.25·1 + 0.125·2	3.5

재귀 결과(2.0, 2.0, 1.0, 3.5)와 **정확히 일치**. 커널이 입력과 무관(LTI)하면 병렬 합성곱이 가능.

연습 4 $\bar{A} = 0.5,\ \bar{B} = 1,\ C = 1,\ x = [2, 1, 0, 3]$ 에서 커널 $K_0 .. K_3$ 을 구하고, 합성곱으로 y_4 를 계산하시오(답은 3.5).

Step 5. 선택성 — Mamba의 심장 (Selective SSM)

> **핵심**: Δ, B, C 를 **입력마다 다르게** 만든다 → 무엇을 기억/무시할지 입력이 결정.

$$\Delta_t = \text{softplus}(\text{Linear}_\Delta(x_t)), \quad B_t = \text{Linear}_B(x_t), \quad C_t = \text{Linear}_C(x_t)$$

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \qquad y_t = C_t h_t + D x_t$$

	LTI (상수)	Selective (입력 의존)
커널	고정 → 합성곱 가능	매번 달라짐 → 합성곱 불가
계산	고속 합성곱	**병렬 스캔** 필요
능력	선택 불가	입력 선택 가능

> **연습 5** 왜 C_t 가 입력에 의존하면 정적 합성곱(컨볼루션)으로 계산할 수 없는지 한 문장으로 설명하시오.

Step 6. 병렬 스캔 — 재귀를 병렬로

> **핵심**: 선형 재귀를 아핀 합성으로 보고, 결합법칙으로 병렬화.

$$h_t = a_t h_{t-1} + b_t, \qquad (a_2, b_2) \circ (a_1, b_1) = (a_2 a_1,\ a_2 b_1 + b_2)$$

쌍의 함수 $z \mapsto az + b$ 로 보면 합성은 결합법칙 성립 → 앞에서부터의 합성(prefix)을 트리로 병렬 계산 $\Rightarrow$ O(L) 작업, O(log L) 깊이.

t	prefix (A_t, B_t)	$h_t = B_t$
1	(0.5000, 2.0000)	2.0000
2	(0.2500, 2.0000)	2.0000
3	(0.1250, 1.0000)	1.0000
4	(0.0625, 3.5000)	3.5000

(Step 3과 같은 예: $a = 0.5$, $b = \bar{B}x = [2, 1, 0, 3]$)

> **연습 6** $a = [0.5, 0.5, 0.5]$, $b = [1, 0, 2]$ 에서 prefix (A_3, B_3) 와 h_3 를 구하시오.

Step 7. Mamba 블록 — 부품의 역할

> **핵심**: "Linear → Conv1d → SiLU → SSM → gate → Linear"

```
x → in_proj → Conv1d(depthwise) → SiLU → SSM(Δ,B,C) → ⦿ SiLU(z) → out_proj → y
```

부품	역할	ViT 대응
`in_proj`	x를 두 갈래(x', z)로	QKV/MLP 확장
`Conv1d`	이웃 토큰 섞기(지역성)	없음(패치 임베딩만)
`SSM`	전역 문맥, O(L)	Self-Attention
`⦿ SiLU(z)`	게이트(통과량 조절)	MLP

연습 7 Conv1d가 없으면 어떤 문제가 생길 수 있는지, 게이트 $\odot \mathrm{SiLU}(z)$ 는 attention의 어떤 역할을 대신하는지 설명하시오.

Step 8. Vision Mamba — 이미지에 적용

> **핵심**: 이미지 → 패치 → 토큰 → **양방향** SSM 스캔.

- 패치화: 224×224, patch 16 → 14×14 = **196** 토큰 (+class = 197)
- 양방향: $y = \mathrm{SSM}_{\mathrm{fwd}}(x) + \mathrm{flip}(\mathrm{SSM}_{\mathrm{bwd}}(\mathrm{flip}(x)))$
- Conv1d + 양방향으로 "순서 의존성"을 완화

이미지는 고정된 순서가 없다 → 단방향 스캔은 패치 순서에 민감 → **양방향이 필수**.

> **연습 8** 448×448 이미지, patch 16일 때 토큰 수와 attention 행렬 원소 수(L^2)를 구하시오.

Step 9. 복잡도 — 왜 빠른가

$$\text{Attention} = O(L^2 D), \qquad \text{SSM} = O(L\,D\,N)$$

L	해상도	Attention ≈ 2L²D	SSM ≈ 3L·d_inner·N	배수
196	224² (D=256)	19.7M	4.8M	4.08×
784	448²	944.1M	57.8M	16.3×

L이 2배 → attention 4배, SSM 2배. 고해상도일수록 격차 폭증. (Vim 논문: DeiT 대비 약 2.8배 빠르고 GPU 메모리 약 86.8% 절감)

연습 9 $L = 196,\ D = 256,\ d_{inner} = 512,\ N = 16$ 에서 attention $2L^2D$ 와 SSM $3Ld_{inner}N$ 을 각각 계산하고 배수를 구하시오.

Step 10. 안전(망막) 적용 — 하이퍼파라미터와 설계

> **핵심**: Mamba 기본값을 그대로 써서 CPU 친화 모델을 구성한다.

`expand=2 → d_inner=2 · d_model, d_state(N)=16, d_conv=4, dt_rank=ceil(d_model/16)`

d_model	d_inner	dt_rank	d_state	d_conv
128	256	8	16	4
256	512	16	16	4
768	1536	48	16	4

설계 메모: 검진 1차는 `normal vs abnormal` 이진 분류부터. 라벨은 소견 문서(`docs/fundus-common-findings-and-red-flags.md`)와 1:1로 맞춘다. **환자 단위 split**(양안 누수 방지) 필수.

연습 10 $d_{model} = 192$ 일 때 d_{inner} 와 dt_rank 를 구하시오.

부록 A. 연습문제 (답은 부록 B)

A1 SSM에서 A 가 영행렬이면 상태가 어떻게 되는가? 어떤 모델과 비슷해지는가?

A2 $A=-2,\ B=0.5,\ \Delta=1$ 에서 $\bar{A}$, 정확한 $\bar{B}$, 근사 $\bar{B}$ (소수 4자리).

A3 $\bar{A}=0.5,\ \bar{B}=1,\ x=[1,2,3]$ 에서 h_1, h_2, h_3.

A4 $\bar{A}=0.5,\ \bar{B}=1,\ C=1,\ x=[2,1,0,3]$ 에서 커널 $K_0..K_3$ 와 $y_1..y_4$.

A5 C_t 가 입력 의존이면 왜 정적 합성곱으로 계산할 수 없는가?

A6 $a=[0.5,0.5,0.5],\ b=[1,0,2]$ 에서 prefix (A_3, B_3) 와 h_3.

A7 Mamba 블록에서 Conv1d가 하는 일과, 게이트가 대신하는 attention의 역할을 쓰시오.

A8 448×448, patch 16 이미지의 토큰 수와 attention 행렬 원소 수.

A9 $L=196, D=256, d_{inner}=512, N=16$ 에서 attention·SSM 비용과 배수.

A10 $d_{model}=192$ 에서 d_{inner}, dt_rank.

A11 (코딩) 아래를 실행해 Step 3과 같은 값이 나오는지 확인하시오.

```
import numpy as np
Abar, Bbar, C = 0.5, 1.0, 1.0
x = [2.0, 1.0, 0.0, 3.0]
h = 0.0
for t, xt in enumerate(x, 1):
    h = Abar*h + Bbar*xt
    print(t, round(h, 4))
```

A12 (서술) attention과 SSM의 "전역 상호작용"이 어떻게 다른지, 그리고 SSM의 정보 병목이 왜 생기는지 3문장 이내로 쓰시오.

부록 B. 정답과 해설

A1 $A = 0$ 이면 상태가 $h_t = \bar{B} x_t$ 로 **직전 입력만** 남고 과거가 사라진다 → 사실상 **메모리 없는 피드포워드(MLP)**에 가까워진다.

A2 $\bar{A} = e^{-2} = 0.1353$. 정확한 $\bar{B} = \frac{e^{-2}-1}{-2} \cdot 0.5 = \frac{0.8647}{2} \cdot 0.5 = 0.2162$. 근사 $\bar{B} = \Delta B = 0.5$.

A3 $h_1 = 0.5 \cdot 0 + 1 \cdot 1 = 1.0$, $h_2 = 0.5 \cdot 1 + 2 = 2.5$, $h_3 = 0.5 \cdot 2.5 + 3 = 4.25$.

A4 $K_k = C\bar{A}^k\bar{B} = 0.5^k \rightarrow K = [1,\ 0.5,\ 0.25,\ 0.125]$. $y_1 = 1 \cdot 2 = 2.0$,
$y_2 = 1 \cdot 1 + 0.5 \cdot 2 = 2.0$, $y_3 = 1 \cdot 0 + 0.5 \cdot 1 + 0.25 \cdot 2 = 1.0$,
$y_4 = 1 \cdot 3 + 0.5 \cdot 0 + 0.25 \cdot 1 + 0.125 \cdot 2 = 3.5$. (재귀 A3/Step3과 동일 값)

A5 합성곱 커널 $K_k = C\bar{A}^k\bar{B}$ 는 C, B 가 상수일 때만 위치에 무관하게 고정된다. C_t, B_t 가 입력마다 바뀌면 커널이 모든 위치에서 달라져 **하나의 고정 커널로 표현할 수 없다**.

A6 $P_1 = (0.5, 1)$, $P_2 = (0.25,\ 0.5 \cdot 1 + 0) = (0.25, 0.5)$,
$P_3 = (0.125,\ 0.5 \cdot 0.5 + 2) = (0.125, 2.25)$. 따라서 $(A_3, B_3) = (0.125,\ 2.25)$, $h_3 = 2.25$.

A7 Conv1d는 이웃 패치를 먼저 섞어 **지역성/근접성**을 주고 SSM의 순서 의존성을 완화한다(블록 내 "위치 감각"). 게이트 $\odot \mathrm{SiLU}(z)$ 는 SSM 출력의 통과량을 입력이 조절해, attention/MLP가 하던 **선택적 정보 전달**을 대신한다.

A8 $448/16 = 28 \rightarrow 28^2 = \mathbf{784}$ 토큰. attention 행렬 원소 $= 784^2 = \mathbf{614{,}656}$.

A9 Attention $= 2 \cdot 196^2 \cdot 256 \approx \mathbf{19.7M}$. SSM $= 3 \cdot 196 \cdot 512 \cdot 16 \approx \mathbf{4.8M}$. 배수 $\approx \mathbf{4.08}\times$.

A10 $d_{inner} = 2 \cdot 192 = \mathbf{384}$. $dt_rank = \lceil 192/16 \rceil = \mathbf{12}$.

A11 출력: `1 2.0` / `2 2.0` / `3 1.0` / `4 3.5` — Step 3과 일치. (`Abar=0.5, Bbar=1` 로 두면 $\bar{B}_t x_t = x_t$.)

A12 Attention은 모든 토큰 쌍을 직접 비교해 전역 상호작용을 만든다. SSM은 고정 크기 상태 $\boldsymbol{h} \in \mathbb{R}^{N}$ 를 통해 정보를 순차 전달하므로, 상태 차원 N이 병목이 되어 매우 긴 문맥의 세부 정보는 압축 과정에서 손실될 수 있다.

참고자료

1. Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model. ICML 2024. arXiv:2401.09417.
2. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752.
3. MedMamba: Vision Mamba for Medical Image Classification. arXiv:2403.03849.
4. LEMUR: Ultra-Lightweight Distilled Retinal Foundation Model Using Mamba (github.com/andregdmitri/LEMUR).
5. 프로젝트 내: `docs/vision-mamba.html`(애니메이션), `app/PLAN-keras-tensorflow.md`(구현 계획).

의료진 학습용 자료입니다. 계산값은 모두 스크립트로 검증했습니다.